

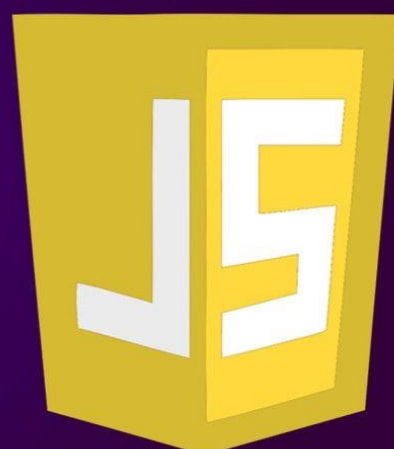
CSS



HTML



JS



COURS DE PROGRAMMATION WEB 2

45 Heures

Par Ir Isaac ILUNGA

Concepteur des Systèmes d'information

PLAN DU COURS

PHASE 0 – Introduction au Web (2–3h)

📖 Objectif : comprendre avant de coder

1. Qu'est-ce que le Web ?
2. Web vs Internet
3. Comment fonctionne un site web ?
 - Client / Serveur
 - Requête / Réponse
4. Types de sites web
 - Site statique
 - Site dynamique
5. Présentation des technologies
 - HTML, CSS, JavaScript
6. Outils de travail
 - Navigateur
 - VS Code
 - Structure d'un projet web

🚀 TP :

- Installer VS Code

- Créer un dossier mon-premier-site
 - Créer index.html et l'ouvrir dans le navigateur
-

☒ PHASE 1 – HTML : Structure du Web (10h)

☞ On construit le squelette

1. Qu'est-ce que HTML ?

- Langage de balisage
- Rôle de HTML

2. Structure d'une page HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Mon site</title>
```

```
</head>
```

```
<body>
```

```
<h1>Bonjour le Web</h1>
```

```
</body>
```

```
</html>
```

3. Balises essentielles

- Titres (h1 à h6)

- Paragraphes (p)
- Liens (a)
- Images (img)
- Listes (ul, ol, li)
- Tableaux (table)
- Formulaires (form, input, button)

4. Sémantique HTML5

- header, nav, main, section, article, footer

✦ TP progressifs

- Page personnelle
- Page école
- Page entreprise fictive



Mini-projet

HTML

👉 Site vitrine statique (3 pages)

📁 PHASE 2 – CSS : Mise en forme et design (12h)

👉 On apprend à rendre beau sans paniquer

1. Introduction au CSS

- Pourquoi CSS ?
 - Où écrire le CSS ?
-

- Inline
- Interne
- Externe (recommandé)

2. Sélecteurs CSS

- Tag
- Classe
- ID

3. Propriétés de base

- Couleurs
- Textes
- Marges & paddings
- Bordures
- Background

4. Box Model (très important)

- content
- padding
- border
- margin

5. Layout

- display

- flexbox (essentiel)
- grid (intro)

6. Responsive design

- Media queries
- Mobile first

✦ TP

- Styliser la page HTML
- Créer un menu
- Créer une page responsive



Mini-projet

CSS

☞ Site vitrine responsive

☒ PHASE 3 – JavaScript : Dynamique et logique (14h)

☞ Ici on relie avec l'algorithmique

1. Introduction au JavaScript

- Rôle de JS
- Où écrire le JS ?
- Console du navigateur

2. Bases du langage

- Variables (let, const)

- Types de données
- Opérateurs
- Conditions (if)
- Boucles (for, while)
- Fonctions

3. Manipulation du DOM

- `document.getElementById`
- `querySelector`
- Événements (click, submit)

4. Interaction utilisateur

- Formulaires
- Validation simple
- Alertes

5. Structures de données (lien avec Algo 2)

- Tableaux
- Objets
- Parcours
- Recherche
- Tri simple

✦ TP

- Calculatrice
- Formulaire dynamique
- Todo list simple



Mini-projet

JS

Application web interactive sans backend

☒ PHASE 4 – Introduction au Web dynamique (6h)

Ouverture vers le vrai Web moderne

1. Qu'est-ce qu'un site dynamique ?
2. Introduction à HTTP
3. Notion d'API
4. JSON
5. fetch()

TP

- Consommer une API publique
- Afficher des données dynamiques

☒ PHASE 5 – Mini-projet final (8–10h)

Objectif : fierté + compétence réelle

Projet proposé :

- Gestion des étudiants
- Gestion des notes
- Mini e-commerce
- Todo app avancée

ÉVALUATION

- Interrogations courtes
- TP notés
- Mini-projets
- Projet final

INTRODUCTION GÉNÉRALE

Le développement web est aujourd'hui un domaine incontournable de l'informatique. Les applications web sont utilisées dans presque tous les secteurs de la société moderne : éducation, administration, commerce, banques, télécommunications, santé, médias et réseaux sociaux. Pour cette raison, la maîtrise des technologies du Web constitue une compétence essentielle pour tout informaticien.

Le cours de **Programmation Web** a pour objectif d'initier les étudiants aux bases du développement des sites et applications web. Il permet de comprendre **comment les pages web sont construites, comment elles sont mises en forme, et comment elles deviennent interactives** grâce aux langages et technologies du Web.

Ce cours est conçu pour des étudiants qui **n'ont jamais suivi un cours de programmation web**. Il commence donc par les notions fondamentales, en expliquant progressivement les concepts, sans supposer de connaissances préalables dans le domaine du Web. Une attention particulière est accordée à la compréhension avant la pratique, afin de permettre aux étudiants d'acquérir des bases solides.

Au cours de ce module, les étudiants apprendront à utiliser les trois technologies principales du Web :

- **HTML**, pour structurer le contenu des pages web ;
- **CSS**, pour la mise en forme et le design des pages web ;
- **JavaScript**, pour ajouter de l'interactivité et de la logique aux pages web.

Le cours adopte une approche **progressive et pratique**. Chaque notion théorique est accompagnée d'exemples et de travaux pratiques, permettant aux étudiants de mettre immédiatement en application ce qu'ils ont appris. Cette méthode vise à développer à la fois la compréhension des concepts et les compétences techniques nécessaires à la création de sites web fonctionnels.

À la fin du cours, l'étudiant sera capable de :

- comprendre le fonctionnement général du Web ;
- créer des pages web structurées en HTML ;
- mettre en forme des pages web avec CSS ;
- ajouter des interactions simples à l'aide de JavaScript ;
- concevoir un site web simple et fonctionnel.

Ce cours constitue une **base fondamentale** pour les modules plus avancés du développement web et prépare les étudiants à l'utilisation de frameworks et technologies web modernes dans la suite de leur formation.

CHAPITRE 1 : INTRODUCTION GENERALE AU WEB

Le Web occupe aujourd'hui une place centrale dans le domaine de l'informatique. Il est utilisé dans presque tous les secteurs : éducation, commerce, administration, communication, santé, finance, etc. Comprendre le fonctionnement du Web et ses technologies de base est donc une compétence indispensable pour tout informaticien.

Avant d'entrer dans la programmation proprement dite (HTML, CSS et JavaScript), il est important de **comprendre ce qu'est le Web, comment il fonctionne et quels outils sont utilisés pour créer un site web**. Ce chapitre constitue une base fondamentale. Il vise à donner une vision claire et progressive du Web, sans complexité inutile, afin de préparer les étudiants à la pratique.

L'objectif principal de ce chapitre est donc de **comprendre avant de coder**.

1. Qu'est-ce que le Web ?

Le **Web** (World Wide Web) est un système qui permet d'accéder à des ressources (pages, images, vidéos, applications) à travers Internet à l'aide d'un navigateur web.

Le Web repose sur :

- Des **pages web**
- Des **liens hypertextes**
- Des **serveurs** qui hébergent les ressources
- Des **navigateurs** qui permettent d'afficher ces ressources

Exemples de sites web : Clubic, Facebook, Wikipedia, YouTube, sites universitaires, sites administratifs, etc.

2. Différence entre le Web et Internet

Il est important de ne pas confondre **Internet** et le **Web**.

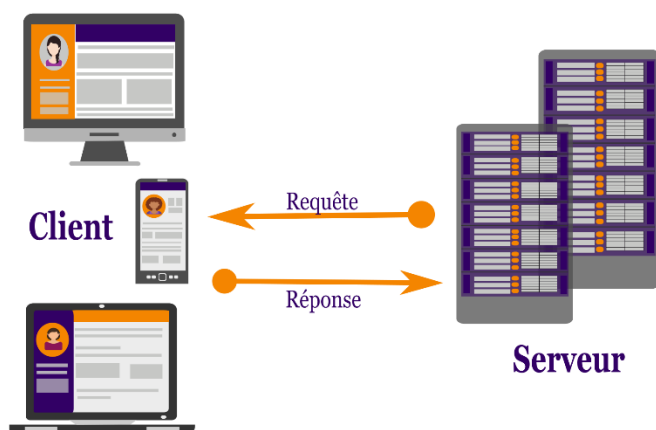
- **Internet** est un réseau mondial d'ordinateurs connectés entre eux.
- **Le Web** est un service qui fonctionne sur Internet.

On peut dire que : **Internet est l'infrastructure**, tandis que **le Web est une application qui utilise cette infrastructure**. Donc le Web est un service qu'offre internet.

Internet permet aussi d'autres services comme :

- Les emails
- Le transfert de fichiers (FTP)
- Les appels en ligne

3. Comment fonctionne un site web ?



Le fonctionnement d'un site web repose sur un modèle appelé **Client – Serveur**.

a) Le client

Le **client** est généralement le navigateur web (Chrome, Firefox, Edge, etc.). Il est utilisé par l'utilisateur pour demander l'affichage d'un site web.

b) Le serveur

Le **serveur** est un ordinateur distant qui contient les fichiers du site web (HTML, CSS, JavaScript, images, données, etc.). Il répond aux demandes du client.

c) Requête / Réponse

1. L'utilisateur entre une adresse web dans le navigateur.
2. Le navigateur envoie une **requête** au serveur.
3. Le serveur traite la requête.
4. Le serveur renvoie une **réponse**.
5. Le navigateur affiche le résultat.

Ce mécanisme est à la base de tous les sites web.

4. Types de sites web

a) Site statique

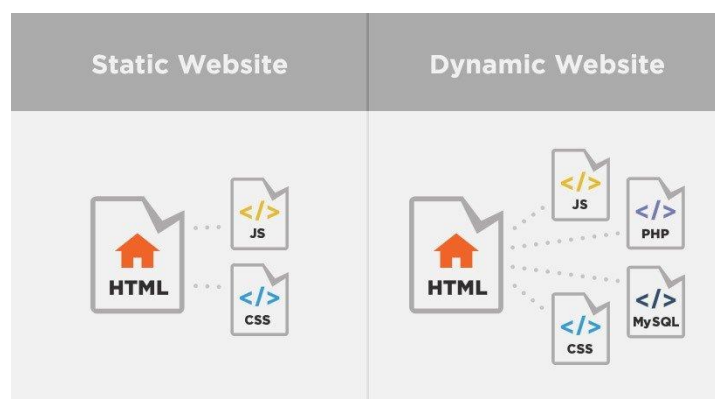
Un site statique est composé de pages fixes.

- Le contenu ne change pas automatiquement.
- Il est écrit principalement en HTML et CSS.
- Exemple : site vitrine, site de présentation.

b) Site dynamique

Un site dynamique peut changer son contenu.

- Il utilise JavaScript et parfois un serveur.
- Il peut interagir avec l'utilisateur.
- Exemple : réseaux sociaux, plateformes de gestion, sites e-commerce.



5. Présentation des technologies du Web

a) HTML (HyperText Markup Language)

HTML permet de **structurer le contenu** d'une page web. Exemples : titres, paragraphes, images, formulaires.

b) CSS (Cascading Style Sheets)

CSS permet de **mettre en forme** les pages web. Exemples : couleurs, tailles, positionnement, design.

c) JavaScript

JavaScript permet de **rendre les pages web interactives**. Exemples : boutons cliquables, formulaires dynamiques, animations, calculs.

Ces trois technologies sont complémentaires et indispensables.

6. Outils de travail

a) Navigateur web

Le navigateur permet :

- D'afficher les pages web
- De tester les sites développés
- D'utiliser les outils de développement

Exemples : Google Chrome, Mozilla Firefox, Microsoft Edge.

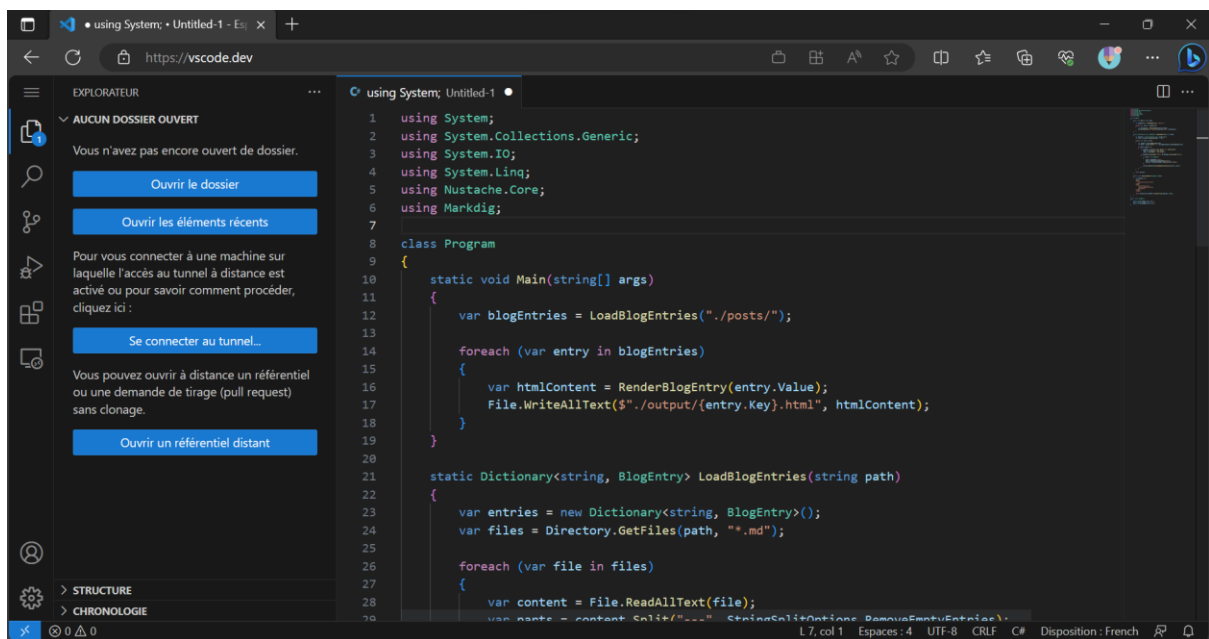


b) Visual Studio Code (VS Code)

VS Code est un éditeur de code moderne utilisé pour écrire :

- HTML
- CSS
- JavaScript

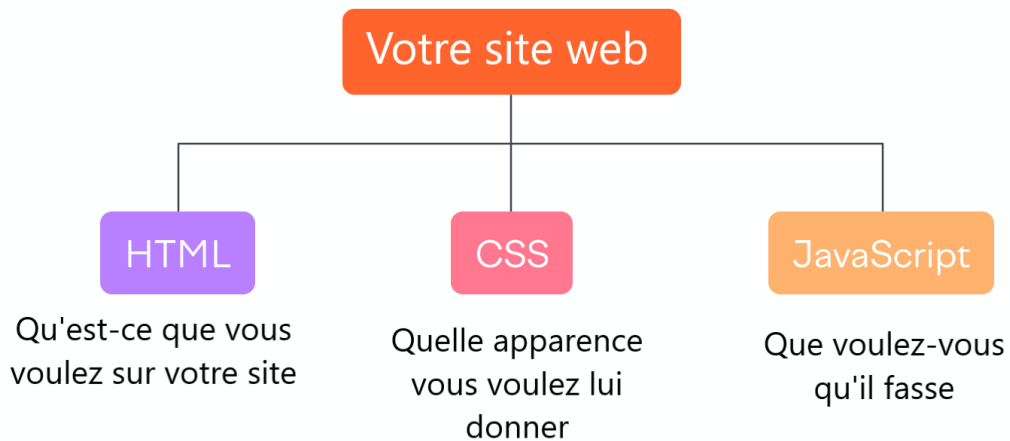
Il est gratuit, rapide et très utilisé dans le monde professionnel.



c) Structure d'un projet web

Un projet web simple peut contenir :

- Un dossier principal du projet
- Un fichier index.html
- Un dossier css
- Un dossier js



TRAVAIL PRATIQUE

Objectif du TP

Mettre en place le premier environnement de travail web.

Instructions

1. Installer Visual Studio Code
2. Créer un dossier nommé **mon-premier-site**
3. Créer un fichier **index.html**
4. Ouvrir le fichier **index.html** dans un navigateur

Ce TP permettra aux étudiants de faire leur **premier contact pratique avec le développement web**.

CHAPITRE 2 : HTML, STRUCTURE DES PAGES WEB

Après avoir compris ce qu'est le Web et comment il fonctionne, nous passons maintenant à la **première technologie fondamentale du développement web** : **HTML**. HTML constitue la base de toute page web. Sans HTML, il n'y a pas de site web.

Dans ce chapitre, nous allons apprendre à **structurer une page web**, c'est-à-dire construire son squelette. Nous verrons les balises essentielles et leur rôle, afin de permettre à l'étudiant de créer des pages claires, bien organisées et compréhensibles.

1. Qu'est-ce que HTML ?

a) HTML : un langage de balisage



HTML signifie **HyperText Markup Language**. HTML n'est pas un langage de programmation, mais un **langage de balisage**. Cela signifie qu'il utilise des **balises** pour organiser et décrire le contenu d'une page web.

Les balises constituent l'élément de base du langage HTML. Chaque balise a une signification que le navigateur connaît et applique au contenu. C'est comme ça que la présentation du document est effectuée.

Une balise est représentée par: le chevron ouvrant + le nom de la balise + le chevron fermant. Exemple `<b>` ou `<body>`.

Il existe deux types de balises: **des balises ouvrantes** et **des balises fermantes**. Elles vont souvent ensemble de telle sorte que la balise ouvrante applique un effet au contenu qui la suit et la balise fermante cesse l'effet en question. La balise fermante se distingue par un slash (/) qui vient juste avant le nom de la balise.

Exemple `</b>` est la balise fermante de `<b>` et `</body>` est la balise fermante de `<body>`.

Néanmoins, il existe des balises ouvrantes qui n'ont pas de balises fermantes associées. On les appelle des **balises auto fermantes ou des balises orphelines**. Ce sont des balises qui s'ouvrent et se ferment au même moment et elles appliquent leur effet à l'endroit où elles sont déclarées (et pas au contenu qui les suit comme les balises qu'on a vu ci haut). Exemple: `<br/>`.

Imbrication des balises

Un contenu peut subir l'effet de plusieurs balises à la fois. Si c'est le cas, alors une règle s'impose: En cas d'imbrication de plusieurs balises, la première balise ouverte est la dernière à fermer. Il faut donc fermer les balises dans l'ordre inverse de leur ouverture.

Le code ressemblera à ceci:

```
<balise1><balise2><balise3>Contenu</balise3></balise2></balise1>
```

Les attributs

La plupart des balises HTML peuvent appliquer des effets différents selon les préférences du développeur. C'est comme si on peut les personnaliser à notre guise. C'est là où les attributs interviennent.

Les attributs sont déclarés dans la balise ouvrante. Ils représentent des paramètres qui personnalisent la balise où ils sont définis et ils possèdent des valeurs. Une balise peut renfermer plusieurs attributs à la fois, chacun avec sa valeur.

Une balise possédant des attributs ressemble à ceci:

```
<balise      attribut1="valeur1"      attribut2="valeur2"
attribut3="valeur3" ...>Contenu</balise>
```

Exemple:

```
<font color="red" face="arial">Bonjour à tous</font>
```

La balise ouvrante applique la couleur rouge et la police Arial au contenu qui la suit qui n'est rien d'autre que le message "Bonjour à tous". La balise fermante cesse l'effet appliqué par la balise ouvrante du même nom.

Les caractères spéciaux

HTML est constitué principalement de balises. Mais ce n'est pas tout car il y a des séquences, autres que les balises, que le navigateur reconnaît et remplace par leurs significations. On appelle ces séquences: **caractères spéciaux**.

Le code HTML des caractères spéciaux commence par le symbole "&" et fini par ";". Par exemple **©** signifie "©". Le tableau suivant liste les caractères spéciaux les plus utilisés en HTML:

CARCTÈRE	CODE HTML
"	";
&	&;
<	<;
>	>;
oe	œ
Espace	 ;
£	£;
©	©;
®	®;
±	±;
μ	µ;
½	½;
Ç	Ç;
æ	æ;

b) Rôle de HTML

Le rôle principal de HTML est de :

- structurer le contenu
- organiser les informations
- donner un sens aux éléments de la page

HTML permet de définir :

- des titres
- des paragraphes
- des images
- des liens
- des formulaires

2. Structure d'une page HTML

Toute page HTML suit une structure de base obligatoire.

```
<!DOCTYPE html>

<html>

  <head>

    <title>Mon site</title>

  </head>

  <body>

    <h1>Bonjour le Web</h1>

  </body>

</html>
```

Explication des balises

- **<!DOCTYPE html>** : indique au navigateur que le document est en HTML5.
- **<html>** : balise principale qui englobe tout le document.
- **<head>** : contient les informations générales (métadonnées) du document.
- **<title>** : définit le titre affiché dans l'onglet du navigateur.
- **<body>** : contient le contenu visible de la page.

3. Balises essentielles

3.1. Balises de formatage de texte sans attributs

Nous allons d'abord commencer par les balises qui ne nécessitent pas d'attributs. A elles seules, elles peuvent formater un texte.

a) Balise **** et ****

La balise **** met le texte qu'elle contient en gras (bold). C'est utile pour les textes importants ou les titres.

Par exemple: ****Ce texte est écrit en gras****

La balise **** met aussi le texte en gras, elle peut remplacer la balise ****, bien que celle-ci est plus populaire.

b) Balise **<u>**

La balise **<u>** (pour underline) souligne le texte qu'elle entoure. C'est utile pour attirer l'attention à un mot ou une phrase.

Par exemple: **<u>**Texte souligné**</u>**

c) Balise **<i>**

La balise **<i>** (pour italic) met le texte en italique. C'est pratique pour les citations ou les notes.

Par exemple: `<i>Texte en italique</i>`

d) Balise `<s>`

La balise `<s>` (pour strike) barre le texte. C'est pratique pour marquer un contenu obsolète ou invalide (par exemple: sur les sites E-commerce, on barre généralement les anciens prix d'articles en promotion, pour mettre en valeur le nouveau prix).

Par exemple: `<s>Texte barré</s>`

Exemple pour récapituler

Supposons que je veux écrire la phrase suivante "HTML est un langage de description de pages Web et non pas un langage de programmation", de telle sorte que "HTML" soit en gras et souligné, "est un langage de description de pages Web" souligné, et "et non pas un langage de" italique et "de programmation" italique et barré.

Le code qui fait ceci peut s'écrire ainsi:

```
<u><b>HTML</b> est un langage de description de pages Web</u>
  <i>et non pas un langage de <s>programmation</s></i>
```

e) Balises `<sup>` et `<sub>`

La balise `<sup>` met un texte en exposant et la balise `<sub>` le met en indice.

Par exemple, le code suivant: `x<sup>2</sup>` donne: x^2

et ce code: `H<sub>2</sub>O` donne H_2O

f) Retour à la ligne: Balise `<br>`

Pour retourner à la ligne à n'importe quel moment, il suffit de déclarer la balise `<br>`. Il s'agit d'une balise orpheline (donc pas de `</br>`).

Exemple:

Ligne 1 `<br />`

Ligne 2 `<br />`

Et ligne 3

g) Commentaire HTML `<!-- -->`

Un commentaire HTML est déclaré comme ceci: `<-- Commentaire -->`. Il est visible dans le code source mais ignoré par le navigateur. Il sert à marquer un bloc de code pour que celui-ci soit facilement trouvé et compris lors de sa prochaine manipulation.

Exemple:

`<!-- Ceci est un commentaire -->`

`<i>Le commentaire n'est pas visible sur le navigateur.</i>`

h) Balise `<pre>`

Nous avons vu précédemment que pour intégrer un retour à la ligne il faut déclarer la balise `<br>`, et pour mettre plusieurs espaces ou tabulations il faut passer par le caractère spécial ` `. Cependant, il existe une méthode très simple pour afficher un contenu tel qu'il a été édité dans le code. C'est à dire, si vous mettez une tabulation dans le code, celle ci sera automatiquement reprise dans le navigateur, et si vous sautez une ligne, ce saut de ligne sera aussi visible sur le navigateur. La solution magique s'appelle la balise `<pre>`.

Exemple:

`<pre>`

Là-haut sur la montagne, l'était un vieux chalet.


```
Murs blancs, toit de bardeaux,  
Devant la porte un vieux bouleau.  
Là-haut sur la montagne, l'était un vieux chalet.  
</pre>
```

3.2. Balises de formatage de texte avec attributs

a) Titres (h1 à h6)

La balise `<h1>` (heading) désigne un titre de premier niveau. Le texte qu'elle entoure est de grande taille et mis en gras. Il est utile pour créer les grands titres d'une page. La balise `<h2>` désigne un titre de deuxième niveau, elle applique le même effet que la balise précédente mais avec une taille de caractères légèrement plus petite. C'est pratique pour les sous-titres. Il existe aussi `<h3>`, `<h4>`, `<h5>` et `<h6>` avec une taille de caractères de plus en plus petit. Les balises de titre permettent de structurer le contenu par importance.

```
<h1>Titre de premier niveau</h1>  
<h2>Titre de deuxième niveau</h2>  
<h3>Titre de troisième niveau</h3>  
<h4>Titre de quatrième niveau</h4>  
<h5>Titre de cinquième niveau</h5>  
<h6>Titre de sixième niveau</h6>
```

- **h1** : titre principal (un seul par page recommandé)
- **h6** : titre le moins important

NB : Vous avez remarqué un retour à la ligne après chaque balise `<h>`? C'est normal. En effet, les balise `<h>` sont de type block. Il y a deux types de balises en HTML block et inline. Les balises block occupent toute la ligne où elles sont déclarées, même si le contenu (comme du texte) n'est pas assez long. C'est le cas par exemple de la balise

`<h1>`. Par conséquent un retour à la ligne est automatiquement appliqué après la balise. Les balises inline occupent juste assez d'espace pour afficher le contenu qu'ils entourent. Il n'y a donc pas de retour à la ligne automatique après ces balises.

b) Paragraphes `<p>`

La balise `<p>` est une balise de type block, c'est à dire qu'elle crée un bloc et engendre automatiquement un retour à la ligne. Elle sert à définir un paragraphe. On peut la doter de l'attribut `align` qui permet d'aligner, à sa guise, le contenu du paragraphe. Les différentes valeurs de l'attribut `align` sont:

- **left**: C'est la valeur par défaut. Elle permet d'aligner le contenu du paragraphe à gauche de la page (ou à gauche du conteneur qui renferme la balise `<p>`).
- **right**: Elle permet d'aligner le contenu du paragraphe à droite de la page (ou à droite du conteneur qui renferme la balise).
- **center**: Elle permet de centrer le paragraphe.
- **justify**: Elle permet de justifier le contenu du paragraphe (prolonge le texte pour qu'il occupe toute la ligne).

```
<h1>Les paragraphes</h1>
```

```
<p align="left">
```

La balise `<p>` permet de définir un paragraphe au sein d'un code

HTML. Il engendre un retour à la ligne avant et après.

```
</p>
```

c) Les conteneurs (ou containers)

On entend par conteneur (ou container) des balises qui peuvent renfermer d'autres éléments comme du textes ou des images. Les conteneurs les plus célèbres sont les balises `<div>` et `<span>`.

Balise <div>

La balise <div> est une balise de type block. Elle permet de définir un conteneur ou un bloc qui contient d'autres éléments. Il sert généralement à mieux diviser la page Web pour placer le bon contenu au bon endroit. Cependant, la balise <div> nécessite des styles CSS pour qu'elle soit dotée de toute sa force.

En HTML cette balise ne sert pas à grand chose si ce n'était que pour aligner du texte ou définir un paragraphe (Elle se comporte à peu près comme la balise <p>). Les attributs dont on peut la doter sont les mêmes que ceux de la balise <p>.

Exemple:

```
<h1>Les conteneurs</h1>
```

```
<div align="left">
```

La balise <div> permet de définir un conteneur de type block au sein

d'un code HTML. Il engendre un retour à la ligne avant et après.

```
</div>
```

Balise

La balise est aussi un container, mais de type inline (elle occupe juste assez d'espace pour afficher ce qu'elle contient et pas toute la ligne comme la balise <div>). Par conséquent elle n'engendre pas de retour à la ligne.

Exemple:

HTML est un langage de description de pages Web.

d) Liens (a)

La balise <a> (pour anchor ou ancre) permet de créer un lien hypertexte pointant vers un autre document ou autre emplacement. Un lien classique est reconnaissable par sa couleur bleue et un style souligné (ces propriétés peuvent être changées à l'aide du langage CSS).

La balise <a> dispose des attributs suivants:

- **href:** désigne le chemin vers la page de destination après avoir cliqué sur le lien. Comme pour l'attribut src de la balise , le chemin peut être absolu ou relatif.

Si par exemple la valeur de l'attribut href vaut "mailto:adresse@mail" alors, en cliquant sur le lien, le logiciel de messagerie par défaut installé chez le client s'ouvrira en l'invitant à écrire un mail à "adresse@mail".

- **target:** désigne où la page de destination sera ouverte. Il peut contenir plusieurs valeurs mais une seule est fréquemment utilisée il s'agit de _blank (n'oubliez pas le souligné avant le mot blank). La valeur _blank signifie que le lien sera ouvert dans une nouvelle fenêtre (ou nouvel onglet). C'est utile pour les liens pointant vers d'autres sites Web. Si l'attribut target n'est pas spécifié, alors la page de destination sera ouverte dans la même fenêtre.
- **title:** décrit un message qui sera affiché dans une infobulle si on survole le lien avec la souris.

Exemple:

```
<a href="http://www.chiny.me" title="Aller vers chiny.me"
target="_blank">
    Lien vers le site chiny.me
</a>
```

e) Images (img)

La balise `<img>` permet d'afficher une image sur le navigateur. Il s'agit d'une balise orpheline (donc pas de `</img>`).

La balise `<img>` peut avoir plusieurs attributs. Les plus importants sont:

- **src**: désigne la source de l'image. La valeur associée correspond au chemin vers elle (ou son emplacement). Le chemin peut être absolu ou relatif. Le chemin absolu représente l'URL de l'image (par exemple: `http://www.chiny.me/images/Logo_Chiny.png`). Le chemin relatif, quant à lui, désigne l'emplacement de l'image à partir de l'emplacement actuel (par exemple: `images/Logo_Chiny.png`).
- **border**: accepte une valeur numérique (1,2...). Cette valeur désigne l'épaisseur de la bordure qui encadre l'image. L'unité utilisée pour la bordure est le pixel (px) mais elle est implicite, seule la valeur numérique est déclarée. Par défaut la bordure est de couleur noire.
- **width et height**: signifient, respectivement, la largeur et la hauteur de l'image. L'unité peut être le pixel (qui est implicite) ou le pourcentage (qu'il faut expliciter à l'aide du symbole universel %). Si par exemple on donne à la largeur la valeur 100%, cela veut dire que l'image occupe toute la largeur de la page (ou du conteneur qui accueille la balise `<img>`).
 - Si width ou height ne sont pas spécifiée alors l'image sera affichée en taille réelle.
 - Si seule la valeur d'une dimension (width ou height) est déclarée, l'autre dimension sera automatiquement recalculée par le navigateur de telle sorte à conserver les proportions originales de l'image.
- **alt**: représente le texte alternatif à l'image. Il accepte comme valeur un mot ou une phrase qui sera affichée à la place de l'image si celle ci n'est pas chargée correctement.
- **title**: il s'agit d'un message qui sera affiché dans une infobulle si on survole l'image avec la souris.

Le code suivant affichera le logo de ce site en forçant la largeur à 200 pixels.

```
<center>
  
</center>
```

f) Listes

🚦 Liste ordonnée (ol)

La balise `<ol>` permet d'avoir des listes ordonnées (ou numérotées). Elle doit être fermée (par `</ol>`) et elle contient une autre balise qui est `<li>`. La balise `<li>` permet de définir une entrée de la liste (Il y'a donc autant de balises `<li>` que d'entrées). La balise `<ol>` peut avoir des attributs dont les plus importants sont:

- **type**: sert à définir le type de marqueur de la liste. Les différentes valeurs de l'attribut sont: 1 pour les chiffres classiques (il s'agit d'ailleurs de la valeur par défaut), A pour l'alphabet majuscule, a pour l'alphabet minuscule, I pour les chiffres romains majuscules et i pour les chiffres romains minuscule.
- **start**: indique la valeur du marqueur du début. Par défaut la liste commence par 1 (si elle est ordonnée). Si vous mettez `start="5"` alors la première entrée sera marquée par 5 et non par 1, la deuxième par 6...

Les noms des balises de bases à déclarer dans un document HTML sont:

```
<ol type="1">
  <li>HTML</li>
```

```
<li>HEAD</li>
<li>BODY</li>
</ol>
```

Liste non ordonnée (ul)

La balise permet de créer des listes non ordonnées. Elle contient aussi la balise qui permet de définir les éléments de la liste.

Tout comme la balise , la balise possède des attributs qui permettent de personnaliser la liste. Le plus important d'entre eux est l'attribut type qui définit le type de marqueur et qui peut avoir une des valeurs suivantes:

- disc: le marqueur est un cercle plein. Il s'agit de la valeur par défaut.
- circle: le marqueur est un cercle creux.
- square: le marqueur est un carré plein.

On reprend l'exemple précédent avec la balise cette fois:

Les noms des balises de bases à déclarer dans un document HTML sont:

```
<ul type="square">
  <li>HTML</li>
  <li>HEAD</li>
  <li>BODY</li>
</ul>
```

g) Tableaux (table)

Les tableaux peuvent servir de grille qui contient des données structurées en ligne et en colonne. Comme par exemple une liste d'articles avec leurs prix. Mais ils peuvent servir également de conteneurs qui renferment d'autres objet HTML afin de structurer sa page Web sous forme de cellules et faciliter ainsi la mise en page.

Pour créer un tableau nous avons besoin, au moins, de trois balises de base: <table>, <tr> et <td>.

Balise <table>

La balise <table> est le conteneur principal. Elle permet de déclarer un tableau et peut accueillir de nombreux attributs dont les plus fréquents sont:

- **border**: définit l'épaisseur de la bordure du tableau (et de ses cellules). Elle est exprimée en pixels (px) mais nous nous contentons de déclarer la valeur sans l'unité. Celle-ci étant implicite et connue par le navigateur.
- **width**: définit la largeur du tableau. Elle est exprimée en pixel ou en pourcentage. Si on entend les pixels alors on met la valeur sans l'unité. Si c'est du pourcentage alors on met la valeur suivie du symbole %.
- **height**: définit la hauteur du tableau en pixel. Mais il vaut mieux ne pas déclarer cet attribut et laisser le tableau s'étirer en fonction de la taille de son contenu.
- **bgcolor**: désigne la couleur de l'arrière-plan du tableau. Cette couleur est exprimée en nom anglais ou en code de couleur.
- **cellspacing**: signifie l'espacement entre les différentes cellules du tableau. Sa valeur est exprimée en pixel (implicite).
- **cellpadding**: définit la marge interne des cellules. Elle est exprimée en pixel (implicite).

NB : Si le tableau est destiné à devenir conteneur d'autres objet HTML. Alors il serait judicieux que les bordures ne soient pas visibles. On parle alors de tableau masqué. L'attribut border aura donc la valeur 0.

Balise <tr>

Même si vous déclarez la balise <table> dans le code, rien ne s'affichera sur le navigateur. En effet, ce qui peut être affiché ce sont les cellules d'un tableau, et une cellule c'est au moins une ligne et une colonne.

La balise `<tr>` permet d'ajouter une ligne à un tableau. Elle est directement déclarée dans la balise `<table>`. Il y a donc autant de balises `<tr>` que de lignes.

La balise `<tr>` peut accueillir des attributs comme `height` ou `bgcolor`, mais la plupart des développeurs (y compris moi) préfèrent la déclarer sans attribut.

Balise `<td>`

La balise `<td>` représente une colonne. C'est elle en fait le vrai conteneur, car c'est elle qui peut accueillir du contenu. Elle est déclarée dans la balise `<tr>`. Si vous voulez avoir un tableau avec une ligne et deux colonnes, alors vous devez déclarer une balise `<tr>` et dans celle ci, deux balises `<td>`.

Si je vous ai conseillé de laisser la balise `<tr>` sans attributs, c'est parce que c'est la balise `<td>` qui doit en avoir. Les principaux attributs qu'on peut déclarer sur la balise `<td>` sont:

- **width**: définit la largeur de la colonne. Elle est exprimé en pixel ou en pourcentage par rapport à la largeur du tableau.
- **height**: définit la hauteur de la colonne en pixel.
- **bgcolor**: désigne la couleur de l'arrière plan de la colonne. Cette couleur est exprimée en nom anglais ou en code de couleur. Si la couleur d'arrière plan n'est pas déclarée, alors la colonne est transparente, et c'est la couleur de l'arrière plan du tableau qui sera visible.
- **align**: permet de spécifier l'alignement des objets contenus dans la colonne. Elle peut avoir une des valeurs: `left` (valeur par défaut), `right`, `center` ou `justify`.
- **valign**: définit l'alignement vertical des objets contenus dans la colonne. Elle peut avoir une des valeurs: `middle` (valeur par défaut), `top` ou `bottom`. Il existe aussi la valeur `baseline` qui signifie la ligne de base (son effet est proche de celle de la valeur `bottom`).

Exemple

Supposons qu'on veut créer un tableau qui affiche la liste du matériel informatique d'une petite entreprise avec leur quantité. Le code que je propose est le suivant:

```
<table border="1" width="100%" cellspacing="0"
cellpadding="6">
  <tr>
    <td width="50%" bgcolor="#000000">
      <font color="#FFFFFF">Type de matériel</font>
    </td>
    <td width="50%" bgcolor="#000000">
      <font color="#FFFFFF">Quantité</font>
    </td>
  </tr>
  <tr>
    <td width="50%">PC de bureau</td>
    <td width="50%">26</td>
  </tr>
  <tr>
    <td width="50%">PC portables</td>
    <td width="50%">32</td>
  </tr>
  <tr>
    <td width="50%">Imprimantes</td>
    <td width="50%">12</td>
  </tr>
</table>
```

Ce qui donne le résultat suivant:

Type de matériel	Quantité
PC de bureau	26
PC portables	32
Imprimantes	12

FUSIONNEMENT DE CELLULES D'UN TABLEAU

Si on suppose qu'on veut créer un tableau qui contient deux lignes, et si la ligne du dessus contient deux colonnes, alors ce qu'on a fait jusqu'ici ne nous permet pas d'avoir un nombre différent de colonnes dans la ligne du dessous. Alors cette dernière ne peut comporter que deux colonnes si on tient à avoir un affichage correct. Néanmoins, il existe des cas (d'ailleurs très nombreux) où on a besoin d'avoir un nombre de cellules différent dans chaque ligne. La solution magique est le fusionnement des cellules.

On peut fusionner les cellules horizontalement ou verticalement. Mais le plus fréquent c'est de fusionner à l'horizontale grâce à l'attribut `colspan`.

L'attribut `colspan`

L'attribut `colspan` est inclus dans la balise de colonne `<td>`. Il indique à celle-ci combien de colonnes il faut fusionner pour en avoir qu'une seule à la fin.

Un exemple sera en mesure de clarifier les idées. Alors, supposons que je veux créer un tableau qui contient deux lignes et chaque ligne contient deux colonnes. Bien entendu, y a rien de nouveau à cela. Voici le code:

```
<table>
  <tr>
    <td>Cellule 1</td>
    <td>Cellule 2</td>
  </tr>
  <tr>
```

```

        <td>Cellule 3</td>
        <td>Cellule 4</td>
    </tr>
</table>

```

Ce code donne le résultat suivant:

Cellule 1	Cellule 2
Cellule 3	Cellule 4

Maintenant on ne veut garder dans la première ligne que la colonne qui contient le texte "Cellule 1". Il est donc évident qu'on va retirer la <td> qui contient le texte "Cellule 2", mais notre tableau sera déformé. Pour éviter ce problème, sur la colonne que l'on veut garder, on va ajouter l'attribut colspan auquel on va donner la valeur 2. Littéralement, ceci veut dire qu'on a fusionné 2 colonnes pour n'en avoir qu'une seule.

Voici le nouveau code:

```

<table>
    <tr>
        <td colspan="2">Cellule 1</td>
    </tr>
    <tr>
        <td>Cellule 3</td>
        <td>Cellule 4</td>
    </tr>
</table>

```

Et voilà le résultat:

Cellule 1	
Cellule 3	Cellule 4

Si vous voulez fusionner 3 colonnes. Supprimez deux d'entre elles, et sur la colonne restante mettez colspan="3" etc etc.

L'attribut rowspan

Si colspan fusionne les colonnes à l'horizontale, rowspan quant à elle, elle les fusionne à la verticale. Le principe est le même: Enlevez la (les) colonnes en plus, et sur celle qui reste mettez rowspan="le nombre de colonnes à fusionner".

Par exemple, avant le fusionnement:

```
<table>
  <tr>
    <td>Cellule 1</td>
    <td>Cellule 2</td>
  </tr>
  <tr>
    <td>Cellule 3</td>
    <td>Cellule 4</td>
  </tr>
</table>
```

Le résultat est:

Cellule 1	Cellule 2
Cellule 3	Cellule 4

Après le fusionnement des cellules 1 et 3 le code devient:

```

<table>
  <tr>
    <td rowspan="2">Cellule 1</td>
    <td>Cellule 2</td>
  </tr>
  <tr>
    <td>Cellule 4</td>
  </tr>
</table>

```

Ce qui donne le résultat suivant:

Cellule 1	Cellule 2
	Cellule 4

h) Formulaires

Un formulaire HTML permet de rendre la page Web plus interactive en la rendant capable de dialoguer avec l'internaute à travers des champs de saisie et boutons. Un formulaire permet à l'internaute de saisir du texte ou de valider un choix ou encore de sélectionner une entrée. Ces informations seront ensuite (dans la plupart des cas) envoyées au serveur pour les traiter. C'est très utile surtout quand il s'agit d'une page Web dynamique.

Balise <form>

La balise <form> délimite les champs du formulaire (zones de texte, boutons...). Elle peut avoir plusieurs attributs dont voici les plus fréquents:

L'attribut name

L'attribut name permet d'identifier le formulaire par un nom. On peut mettre n'importe quel nom (à conditions qu'il ne contienne que les caractères conventionnels

comme les lettres et les chiffres). Le nom du formulaire est unique. En effet, nous pouvons déclarer plusieurs formulaires dans la même page, pour les distinguer les uns des autres il suffit de leur donner des noms différents.

L'attribut method

L'attribut method permet de spécifier la méthode à utiliser pour envoyer le formulaire au serveur. En effet, l'objectif d'un formulaire c'est de transférer les données saisies par le client au serveur (on dit généralement poster ou soumettre le formulaire). En HTML, il existe deux méthodes pour envoyer un formulaire: GET et POST.

La méthode GET

La méthode GET (method="get") est la valeur par défaut si on ne précise aucune valeur pour l'attribut method. Cette méthode permet d'envoyer les données du formulaires à travers l'URL en utilisant les symboles ? pour préciser la suite de données et & pour séparer les données entre elles.

Pour mieux comprendre, imaginons que sur ce site il existe une page nommée infos.html qui renferme un formulaire utilisant la méthode GET et qui contient deux zones de textes nommées respectivement ville et pays. Au chargement du formulaire, le client à saisie dans les deux champs les valeurs Marrakech et Maroc. Quand le client ordonnera l'envoi du formulaire, les données saisies seront envoyées via l'URL et celui ci aura la forme suivante:

<http://www.monsitefr/infos.html?ville=Marrakech&pays=Maroc>

La méthode POST

La méthode POST (method="post") permet d'envoyer les données du formulaire au serveur à travers une entête. Pour simplifier, on va dire que les données et la page ne font qu'un. Les informations ne seront donc pas visibles sur l'URL. La méthode POST permet aussi de poster un volume de données plus important que celui

transmis par GET. D'ailleurs on peut même poster un fichier à travers un formulaire via cette méthode. C'est ce que l'on appelle le "Upload" (C'est comme si vous "uploadez" une vidéo sur Youtube, ou votre photo de profil sur Facebook).

NB : Il ne faut en aucun cas penser que le fait que les données ne sont pas visibles sur l'URL grâce à la méthode POST les rend sécurisées. C'est loin d'être le cas!

Que contient donc la balise `<form>`? Elle contient ce que l'on appelle les champs de formulaire. Ce sont des objets comme des zones de texte ou des boutons. Nous allons maintenant voir comment intégrer ces champs.

Balise `<input>`

La zone de texte est le champ de formulaire le plus célèbre. Il permet d'écrire un texte sur une seule ligne (comme le nom ou le login par exemple). Pour créer une zone de texte on fait appel à la balise `<input>`. Il s'agit d'une balise orpheline (pas de `</input>`).

NB : Notez que la plupart des champs de formulaire existants sont déclarés aussi à l'aide de la même balise `<input>`. Pour distinguer une zone de texte d'un bouton par exemple, on fait appel à l'attribut `type`. Détaillons les attributs les plus utilisées pour les inputs.

Attribut `type`

L'attribut `type` permet de définir le type de champ à intégrer. Voici les valeurs à mettre pour cet attribut et qui permettent d'avoir les champs les plus fréquents:

- **text:** Permet d'avoir une zone de texte normale (adapté pour les noms, emails, login...)
- **password:** Permet d'avoir une zone de mot de passe où les caractères sont masqués.

- **radio**: Pour créer un bouton radio. Il s'agit d'un petit cercle qu'on peut cocher et qui permet de faire un choix unique. Le fait de cocher un autre bouton radio décoche le premier.
- **checkbox**: Permet de créer une case à cocher. C'est un petit carré qu'on peut cocher ou décocher. Il est destiné à faire des choix multiples.
- **file**: Permet de créer le champ de chargement de fichier (comme une pièce jointe d'un email).
- **hidden**: Pour créer un champ caché. C'est comme une zone de texte mais qui n'est pas visible sur le navigateur. Il sert à stocker provisoirement des données jusqu'à la phase de traitement.
- **button**: Permet de créer un bouton simple. A la base, ce bouton ne fait aucune opération. Il faut donc le programmer (généralement avec Javascript).
- **reset**: Permet de créer un bouton d'annulation. Si le formulaire contient des champs qu'on a déjà saisis, ce bouton permet de tout initialiser.
- **submit**: C'est le bouton d'envoi. C'est lui qui permet de poster le formulaire vers la page définie dans l'attribut action de la balise <form>.

Attribut name

L'attribut name permet de donner un nom au champ de formulaire. Le nom sert d'identifiant du champ et doit être unique.

Attribut value

L'attribut value sert à définir la valeur par défaut d'un champ. S'il s'agit d'une zone de texte, le fait de déclarer l'attribut value force le navigateur à initialiser ce champ avec le texte faisant office de valeur de l'attribut. Si le champ est un bouton, alors le texte de l'attribut value constitue l'étiquette (ou label). C'est le texte visible sur le bouton. Pour les boutons radio ou les cases à cocher, le texte de l'attribut value n'est pas visible sur le navigateur mais il est très utile (si on fait appel à des programmes comme Javascript ou PHP).

Attribut size

L'attribut `size` contient un entier comme valeur. Il définit la largeur du champ en caractères. Par défaut, une zone de texte mesure 20 caractères (cela veut dire qu'on peut voir jusqu'à 20 caractères à la fois même si le texte est plus grand).

Attribut `tabindex`

L'attribut `tabindex` permet de définir l'ordre de tabulation. En pratique, si on veut saisir un grand formulaire, on se déplace d'un champ à un autre en appuyant sur la touche "tabulation" du clavier. Par défaut le curseur se déplace dans l'ordre où sont déclarés les champs. On peut rompre cet ordre en définissant les valeurs de `tabindex`. La valeur est un entier qui commence de 1. Le curseur se déplacera dans l'ordre croissant de `tabindex`.

Attribut `checked`

L'attribut `checked` est un attribut booléen. Explicitement il est déclaré comme ceci: `checked="true"`, mais il suffit de déclarer `checked` tout seul pour que le navigateur sache qu'il est activé. Cet attribut s'applique uniquement sur les boutons radio (`type="radio"`) et les cases à cocher (`type="checkbox"`). S'il est déclaré alors le bouton est automatiquement coché au chargement de la page.

Il existe deux autres champs de formulaire mais qui ne sont pas déclarés à l'aide de la balise `<input>`. Ce sont les espaces de texte et les listes de sélection.

Balise `<textarea>`

La balise `<textarea>` permet de définir un espace de texte. Un espace de texte est une grande zone de texte qui permet d'écrire des paragraphes entiers.

Les attributs `name` et `tabindex` sont aussi applicables sur l'espace de texte mais, en plus, il dispose des attributs suivants:

- **`cols`**: définit la largeur en caractères de l'espace de texte. Elle désigne combien de caractère une ligne peut-elle contenir. Le fait de dépasser la largeur définie entraîne un retour automatique à la ligne.

- **rows**: définit la hauteur de l'espace de texte. La hauteur désigne combien de ligne on peut voir à la fois. Si le texte dépasse le nombre de lignes défini, alors une barre de défilement qui permet d'atteindre le reste des lignes apparaît.

Balise <select>

La balise <select> permet de déclarer une liste de sélection (ou liste déroulante). Elle dispose également des attributs name et tabindex. Pour remplir cette liste avec les valeurs (appelées options) on doit déclarer une autre balise à l'intérieur, c'est la balise <option>. Dans la balise <option> on déclare le mot à afficher (en tant qu'élément de la liste).

La balise <option> dispose d'un attribut booléen du nom de selected. Si cet attribut est déclaré, c'est l'option qui le contient qui sera automatiquement sélectionnée au chargement de la page.

Exemple de formulaire d'inscription

```
<form name="inscription" method="post" action="">
  Civilité<br />
  <input type="radio" name="civilite" /> Mlle<br />
  <input type="radio" name="civilite" /> Mme<br />
  <input type="radio" name="civilite" /> M.<br />
  Nom et prénom<br />
  <input type="text" name="nom" value="" /><br />
  Email<br />
  <input type="text" name="email" value="" /><br />
  Login<br />
  <input type="text" name="login" value="" /><br />
  Mot de passe<br />
  <input type="password" name="pass" value="" /><br />
  Vous être<br />
```

```

<select name="niv">
  <option>Etudiant</option>
  <option>Fonctionnaire</option>
  <option>Employé au secteur privé</option></select>
<br />
<input type="button" name="envoyer" value="S'inscrire" />
</form>

```

Ce qui donne le résultat suivant:

Civilité

☐ Mlle

☐ Mme

☐ M.

Nom et prénom

Email

Login

Mot de passe

Vous être

Etudiant ▼

S'inscrire

4. Sémantique HTML5

Examinons maintenant les balises sémantiques (ou conteneurs HTML5) les plus utilisés.

a) Balise <header>

Comme son nom l'indique la balise <header> est la balise d'entête (Il ne s'agit pas de la balise <head>. Là on parle de conteneurs, donc ce sont des balises intégrées dans la balise <body>). Si par exemple le site appartient à une entreprise,

alors la balise `<header>` contiendra des informations comme le logo, l'enseigne ou le slogan.

Il ne faut pas s'attendre à ce que le navigateur place la balise `<header>` en haut de la page. Elle sera affichée là où elle a été déclarée. Il faut donc faire en sorte de la mettre en haut de la page en la déclarant en premier ou en utilisant des styles CSS.

b) Balise `<footer>`

La balise `<footer>` est le pied de page. Elle doit normalement être placée en bas de celle ci ou en bas d'un bloc. Elle contient des informations en guise de conclusion. Elle peut aussi contenir des liens du bas de page, les copyrights ou des mentions spécifiques comme les termes légaux ou les conditions d'utilisation...

c) Balise `<nav>`

La balise `<nav>` renferme les liens du menu de navigation principal du site Web. On peut la placer n'importe où sur notre page, mais il est fréquent que le menu principal soit placé sur les cotés ou sur le bord de la page d'une manière assez visible pour faciliter la navigation.

d) Balise `<section>`

La balise `<section>` constitue un bloc principal qui traite le même sujet ou contient des informations de même nature. C'est généralement la partie dont le contenu change d'une page à une autre. Une section peut contenir sa propre entête (`<header>`) et son propre pied de page (`<footer>`).

e) Balise `<aside>`

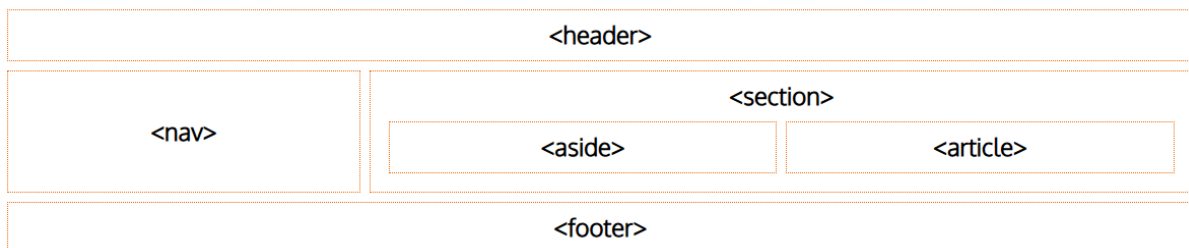
La balise `<aside>` renferme du contenu supplémentaire à un bloc. Elle peut être placée dans la balise `<section>` ou déclarée à part. Le contenu qu'elle peut inclure

peut représenter des liens annexes à un sujet traité dans une section, la définitions d'un terme apparu avant, les références d'une publication...

f) Balise <article>

La balise <article> est destinée à accueillir des informations externes au site Web. En effet, une page ne contient pas toujours que notre propre contenu. On peut, au choix, mettre des blocs qui contiennent des données provenant d'autres sites web comme la météo, les flux RSS, les annonces ou la publicité. Le meilleur endroit pour mettre ce contenu c'est dans la balise <article>. De cette manière le navigateur et les outils de recherche de et d'indexation savent que ce sont des données qui proviennent d'ailleurs.

Voici un exemple de répartition des balises sémantique dans un document HTML5:



g) Balise <main> : contenu principal

Exemple :

```

<header>

  <h1>Mon site</h1>

</header>

<nav>

```

```
    <a href="#">Accueil</a>

</nav>

<main>

    <section>

        <article>

            <p>Contenu principal</p>

        </article>

    </section>

</main>

<footer>

    <p>© 2025</p>

</footer>
```

TRAVAUX PRATIQUES

TP 1 : Page personnelle

Créer une page contenant :

- Nom
- Photo
- Description

TP 2 : Page école

Créer une page présentant :

- Nom de l'école
- Filières

- Coordonnées

TP 3 : Page entreprise fictive

Créer une page web présentant une entreprise imaginaire avec :

- En-tête
- Sections
- Pied de page

Objectif du chapitre

À la fin de ce chapitre, l'étudiant doit être capable de :

- comprendre la structure d'une page HTML
- utiliser correctement les balises essentielles
- créer une page web statique bien structurée

TP 4 : Coder cette page :



[Accueil](#) | [Présentation](#) | [Activités](#) | [Contact](#)

Mon premier exercice

Cet exercice permet de mettre en pratique les balises que nous avons vu jusqu'ici.

N'attendez pas à ce que le design soit épatant! En effet, il reste encore du chemin à faire avant de réussir une page Web parfaite avec un design **agréable**.

Webmaser © 2015

CHAPITRE III. CSS : MISE EN FORME ET DESIGN DES PAGES WEB

Après avoir appris à structurer une page web avec HTML, nous abordons maintenant la **mise en forme des pages web grâce au CSS**. Une page HTML sans CSS fonctionne, mais elle reste simple et peu attrayante. CSS permet de rendre les pages **belles, lisibles et agréables à utiliser**.

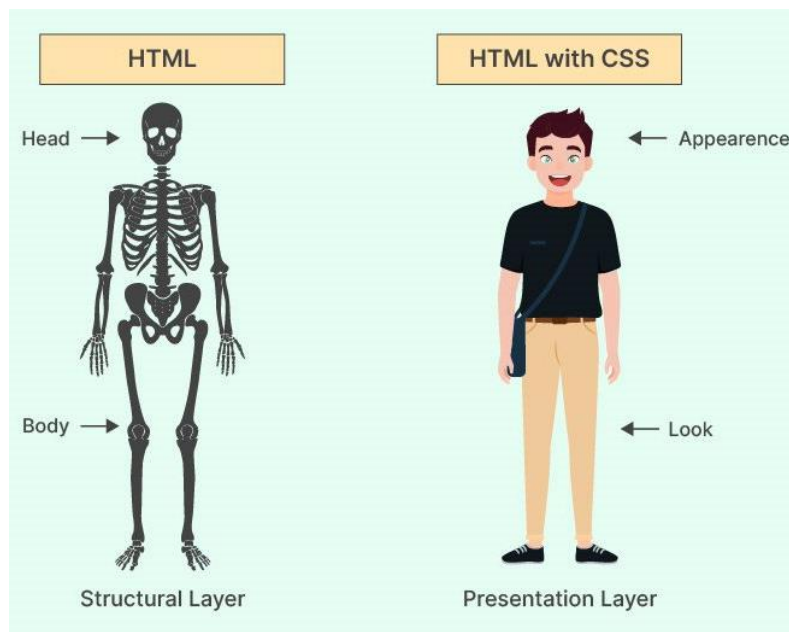
Dans ce chapitre, nous allons apprendre les bases du CSS **pas à pas**, sans complexité, afin de permettre à l'étudiant de styliser progressivement ses pages web.

III.1. INTRODUCTION AU CSS

a) Qu'est-ce que le CSS ?

CSS signifie **Cascading Style Sheets** (feuilles de style en cascade). CSS est un langage de style qui permet de **définir l'apparence** d'une page web :

- couleurs
- tailles
- positions
- disposition des éléments



b) Pourquoi utiliser le CSS ?

CSS permet de :

- séparer le contenu (HTML) de la présentation (CSS)
- rendre les pages plus lisibles et professionnelles
- adapter l'affichage aux différents écrans
- modifier l'apparence sans toucher au HTML

c) Où écrire le CSS ?

Il existe trois façons d'écrire du CSS.

1) CSS en ligne (inline)

Le style est écrit directement dans la balise HTML.

```
<p style="color: red">Texte rouge</p>
```

⚠ Peu recommandé (difficile à maintenir).

2) CSS interne

Le CSS est écrit dans la balise <style> du fichier HTML.

```
<head>
  <style>
    p {
      color: blue;
    }
  </style>
</head>
```

3) CSS externe (recommandé)

Le CSS est écrit dans un fichier séparé .css.

```
<link rel="stylesheet" href="style.css" />
```

Avantages :

- code plus propre
- réutilisable
- professionnel

III.2. SELECTEURS CSS

Les sélecteurs définissent les éléments sur lesquelles s'applique un ensemble de règles CSS.

III.2.1. Les sélecteurs simples

a) Les sélecteurs de type ou de balise (tag)

Ce sélecteur simple permet de cibler les éléments qui correspondent au nom indiqué. Syntaxe : `nomélément` Exemple : `p` permettra de cibler n'importe quel élément `<p>`.

```
p {
  color: black;
}
```

👉 Applique le style à toutes les balises `<p>`.

b) Les sélecteurs de classe (class)

Ce sélecteur simple permet de cibler les éléments en fonction de la valeur de leur attribut class. Syntaxe : `.nomclasse` Exemple : `.index` permettra de cibler n'importe quel élément qui possède la classe `index` (vraisemblablement définie avec un attribut `class="index"`).

Exemple HTML :

```
<p class="important">Texte important</p>
```

CSS :

```
.important {
  color: red;
}
```

☞ Une classe peut être utilisée plusieurs fois.

c) Les sélecteurs d'identifiant (id)

Ce sélecteur simple permet de cibler un élément d'un document en fonction de la valeur de son attribut id. Dans un document, il ne doit y avoir qu'un seul élément pour un identifiant donné. Syntaxe : #valeurid Exemple : #toc permettra de cibler l'élément qui possède l'identifiant toc (vraisemblablement défini avec un attribut id="toc").

Exemple HTML :

```
<h1 id="titre">Bienvenue</h1>
```

CSS :

```
#titre {
  color: green;
}
```

☞ Un ID doit être unique.

d) Le sélecteur universel

Ce sélecteur permet de cibler tous les nœuds d'un document. Il existe également une variante pour ne cibler qu'un seul espace de noms et une variante pour

cibler tous les espaces de noms. Syntaxe : `* ns|* *|*` Exemple : `*` permettra de cibler tous les éléments du document. Ex :

```
* {
  color: #888888;
  font-size: 10pt;
  font-family: verdana, arial, sans-serif;
}
```

e) Les sélecteurs d'attribut

Ce sélecteur simple permet de cibler des éléments d'un document en fonction de la valeur d'un de leurs attributs. Syntaxe : `[attr]` `[attr=valeur]` `[attr~=valeur]` `[attr|=valeur]` `[attr^=valeur]` `[attr$=valeur]` `[attr*=valeur]` Exemple : `[autoplay]` permettra de cibler tous les éléments qui possède l'attribut `autoplay` défini (quelle que soit sa valeur). Ex :

```
img[title] {
  opacity: 0.7;
  filter: alpha(opacity=70);
}

input[type="text"] {
  border: solid 1px blue;
}

[autoplay] {
  color: green;
}
```

III.2.2. Les combineurs

a) Les sélecteurs de voisin direct

Le combinateur '+' permet de sélectionner les nœuds qui suivent immédiatement un élément donné. Syntaxe : A + B Exemple : div + p permettra de cibler n'importe quel élément <p> qui suit immédiatement un élément <div>.

```
div + p {  
  margin: auto;  
}
```

b) Les sélecteurs de voisins

Le combinateur '~' permet de sélectionner les nœuds qui suivent un élément et qui ont le même parent. Syntaxe : A ~ B Exemple : p ~ span permettra de cibler les éléments qui suivent (immédiatement ou non) un élément <p> et qui ont le même élément parent.

```
p ~ span {  
  font-family: 'Courier New', Courier, monospace;  
}
```

c) Les sélecteurs d'éléments enfants

Le combinateur '>' permet de sélectionner les nœuds qui sont des enfants directs d'un élément donné. Syntaxe : A > B Exemple : ul > li permettra de cibler tous les éléments qui sont directement situés sous un élément .

```
ul > li {  
  border: solid 1px black;  
}
```

d) Les sélecteurs d'éléments descendants

Le combinateur " " (espace) permet de sélectionner les nœuds qui sont des descendants (pas nécessairement des enfants directs) d'un élément donné. Syntaxe

: A B Exemple : `div span` permettra de cibler n'importe quel élément `<span>` situé à l'intérieur d'un élément `<div>`.

```
div span {
  color: black;
  font-weight: 600;
}
```

e) Cibler plusieurs éléments à la fois

Pour désigner plusieurs éléments à la fois, il suffit de les déclarer tous séparés par une virgule. Exemple:

```
header, footer, nav, section {
  margin: 0;
  border: none;
}
```

Dans ce cas, les balises `<header>`, `<footer>`, `<nav>` et `<section>` n'auront aucune marge ni aucune bordure.

III.2.3. Les Pseudo-classes

Une pseudo-classe est un mot clé préfixé par deux points (:) qui s'ajoute à un sélecteur CSS pour appliquer un style à un élément dans un cas particulier. L'utilisation des pseudo-classes est très populaire sur les liens hypertextes qui changent de décor selon si ils sont nouvellement affichés, déjà visités ou survolés... Or, leur utilisation s'étend aux autres objets HTML tel que les images, les conteneurs, les listes...

Voici la liste des pseudo-classes les plus utilisées en CSS:

a) Pseudo-classe :link

La pseudo-classe :link désigne un lien hypertexte dont la page cible (celle spécifiée dans son attribut href) n'a pas encore été visitée. Elle peut aider les internautes à distinguer les liens qu'ils n'ont pas encore exploré.

b) Pseudo-classe :visited

A l'inverse de link, la pseudo-classe :visited désigne un lien hypertexte dont la page cible a déjà été visitée. C'est un genre d'historique en quelque sorte.

c) Pseudo-classe :active

La pseudo-classe :active désigne un lien hypertexte sélectionné. Il s'agit de l'instant où le visiteur clique sur le lien, si celui-ci reste affiché sur la page (cas où la page est figée après le clic, ou le lien est ouvert dans une nouvelle fenêtre) on peut voir l'effet de cette pseudo-classe.

d) Pseudo-classe :hover

La pseudo-classe :hover désigne un objet survolé par le curseur de la souris, il peut être un hyperlien ou n'importe quel autre objet. **Exemple :**

```
a {  
    font-weight: bold;  
    color: #ee6600;  
    text-decoration: none;  
}  
a:hover {  
    color: #000088;  
    text-decoration: underline;  
}  
input[type="text"] {  
    border: solid 1px #cccccc;
```



```

color: #888888;
padding: 10px;
border-radius: 4px;
}
input[type="text"]:focus {
border: solid 1px #ee6600;
color: #888888;
outline: none;
}

```

III.3. PROPRIETES CSS DE BASE

III.3.1. Couleurs de textes et propriétés agissant sur le fond

a) Couleur de texte

La propriété `color` permet de spécifier la couleur à appliquer à un texte. Le texte peut être placé dans n'importe quelle balise (`<div>`, `<span>`, `<header>`...).

Gestion des couleurs en CSS

Pour décrire une couleur en CSS, il existe plusieurs méthodes:

- **Nom de la couleur en anglais** : A l'instar de HTML, on peut décrire une couleur par son nom. Or, ce n'est pas très pratique comme méthode car peu de couleurs ont réellement de noms, alors qu'ils en existent plus de 16 millions en tout.
- **Code hexadécimal** : Comme pour le HTML, on peut définir une couleur par le code hexadécimal.
- **Code décimal (ou RGB)** : En CSS, une autre méthode pour définir les couleurs existe. Il s'agit d'indiquer le code décimal de la couleur. Le principe reste le même que pour le codage hexadécimal, mais les codes sont exprimés à la base décimale dont les nombres seront, cette fois, compris entre 0 et 255. A titre

d'exemple, le code du rouge est : `rgb(255,0,0)`. Les valeurs dans les parenthèses représentent respectivement les couleurs primaires rouge, vert et bleu.

- **Code RGBA** : Depuis la version 3 de CSS, une nouvelle méthode pour décrire les couleurs est apparue. Il s'agit de RGBA. Cette notation permet de prendre en compte le degré de transparence (le A pour Alpha). Les trois premiers paramètres de RGBA désignent les couleurs primaires et le quatrième indique le degré de transparence de la couleur appliquée. Il est compris entre 0 (pour totalement transparent) et 1 (pour totalement opaque), et peut avoir deux chiffres après la virgule. Par exemple, `rgba(255,0,0,0.5)` désigne un rouge translucide.
- **Le mot clé inherit** : Le mot clé inherit permet à une balise d'hériter le style de la balise dans laquelle elle est incluse. Dans ce cas c'est la couleur qui sera héritée.

Exemple de code CSS:

```
h1 {  
  color: orange;  
}  
  
div {  
  color: #888888;  
}  
  
span {  
  color: rgb(0, 0, 255);  
}  
  
footer {  
  color: rgba(0, 0, 0, 0.5);  
}
```

Ce code applique la couleur orange sur les textes contenus dans les balises `<h1>`, gris pour les `<div>`, bleu pour les `<span>` et noir translucide pour les `<footer>`.

NB : Si vous voulez insérer un commentaire au milieu du code CSS utilisez la notation suivante: `/* votre commentaire */`.

b) Propriétés d'arrière-plan

CSS agit sur les arrières plans en leur appliquant différents style dont voici les plus utilisés:

Propriété background-color

La propriété background-color permet d'appliquer une couleur d'arrière plan à l'objet HTML souhaité. Pratiquement, tous les objets HTML peuvent avoir un arrière plan. La valeur de cette propriété correspond à la couleur que l'on peut décrire par l'une des 4 méthodes expliquées dans le paragraphe précédent.

Si on souhaite colorer le fond de la page Web en entier alors on utilise le sélecteur de balise body:

```
body {  
    background-color: #cccccc;  
}
```

Ce code applique un gris clair à l'arrière-plan du document.

NB : Notez que le point virgule à la fin du style est facultatif car il sert principalement à séparer deux styles qui se suivent.

Propriété background-image

La propriété background-image sert à insérer une image en arrière plan de l'objet souhaité. en utilisant la syntaxe suivante: background-image:url("chemin_de_l_image"). Le chemin de l'image peut être absolu ou relatif à partir de l'emplacement où est déclaré le style.

Il faut noter que l'image est appliquée en arrière plan avec sa taille réelle. Si elle est plus grande que l'objet qui l'accueille alors elle sera tronquée et si elle est plus petite alors elle sera répétée jusqu'à ce que tout le fond de l'objet soit couvert.

```
body {  
    background-image: url("images/footerlogo.png");  
}
```



🚦 Propriété background-repeat

La propriété background-repeat permet de contrôler la répétition de l'image au cas où celle-ci est plus petite que l'objet qui l'accueille. On peut lui donner les valeurs suivantes :

- repeat: C'est la valeur par défaut. Elle permet de répéter l'image sur la largeur et la hauteur de l'objet.
- repeat-x: Sert à répéter l'image seulement sur la largeur de l'objet.
- repeat-y: Sert à répéter l'image seulement sur la hauteur de l'objet.
- no-repeat: Avec cette valeur, l'image est appliquée une seule fois sans répétition. Par défaut elle sera placée sur le coin haut à gauche de l'objet.

Exemple:

```
body {  
    background-image: url("images/footerlogo.png");  
    background-repeat: repeat-x;  
}
```



➤ Propriété background-position

La propriété background-position permet de placer l'image de l'arrière plan à l'endroit souhaité de l'objet. Par défaut, la première image qui s'applique en arrière-plan est placée en haut à gauche, puis les autres images la suivent pour combler tout l'espace disponible.

La valeur que l'on peut donner à cette propriété peut être exprimé en pourcentage (%) ou par les mots clé: top, center, bottom, left et right. Par exemple si on souhaitait placer l'image d'arrière plan au juste milieu de l'objet, la propriété CSS ressemblerait à ceci: background-position:center center ou background-position:50% 50%. Nous avons exprimé deux valeurs séparées par un espace, une pour la largeur et l'autre pour la hauteur.

Exemple:

```
body {
  background-image: url("images/footerlogo.png");
  background-repeat: no-repeat;
  background-position: 50% 50%;
}
```

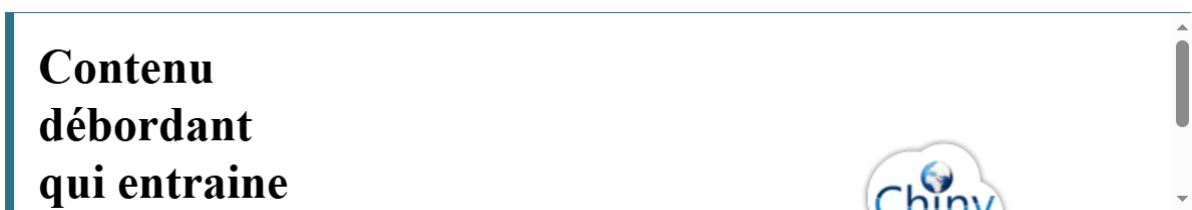


➤ Propriété background-attachment

La propriété background-attachment permet de fixer l'arrière-plan de l'objet si celui là contient une barre de défilement. Elle peut avoir deux valeurs: scroll qui est la valeur par défaut et qui laisse défiler l'arrière plan avec l'avant plan si le client entraîne la barre de défilement qui apparaît sur l'objet et fixed qui permet de figer l'arrière plan pendant que l'avant plan glisse en dessus.

Pour mieux comprendre à quoi cela ressemble on va exécuter le code suivant:

```
body {
  background-image: url("images/footerlogo.png");
  background-repeat: no-repeat;
  background-position: 50% 50%;
  background-attachment: fixed;
}
```



➤ Propriété background-size

La propriété background-size permet de modifier la taille de l'image appliquée en arrière plan d'un container (ou de la page entière). Par défaut, l'image est appliquée avec sa taille réelle. Ce qui pose parfois un problème d'affichage, car soit elle est trop grande ou trop petite par rapport au container qui l'accueille.

La propriété background-size peut accepter deux valeurs; la largeur et la hauteur. Ces valeurs peuvent être exprimées en pixel ou en pourcentage. Cependant, la valeur "cover" semble être trop pratique car elle permet d'adapter l'image au container de telle sorte à en exploiter l'espace maximal.

Appliquons la valeur cover à background-size sur l'image de l'exemple:

```
body {  
    background-image: url("images/footerlogo.png");  
    background-size: cover;  
}
```



🚦 La sous propriété linear-gradient

La sous propriété linear-gradient s'applique à la propriété background pour générer un arrière-plan en dégradé de couleur. Elle possède trois paramètres:

- **Direction du dégradé:** qui peut avoir les valeurs comme to bottom (vers le bas), to left (vers la gauche), to left bottom (incliné vers la gauche et vers le bas) etc... Par défaut elle vaut la valeur to bottom.
- **Couleur de début:** Précise à partir de quelle couleur le dégradé sera engendré. La couleur est spécifiée grâce à l'une des 4 méthodes déjà vues.
- **Couleur de fin:** Précise la deuxième couleur du dégradé.

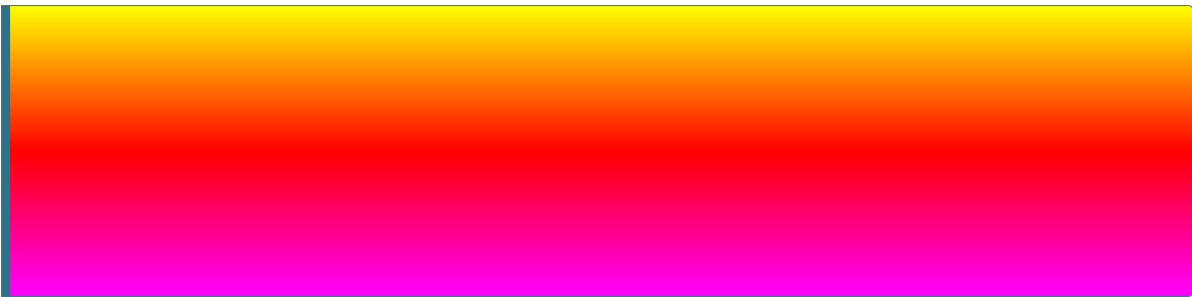
Entre la couleur de début et la couleur de fin le navigateur générera des nuances assurant un passage progressif entre les deux couleurs.

Exemple:

```
body {  
    background: linear-gradient(to bottom, #dddddd, #ffffff);  
}
```



```
body {
  background: linear-gradient(to bottom, #ff0, #f00, #f0f);
}
```



```
body {
  background: linear-gradient(to bottom, #ff0 0%, #f00 30%, #f0f
100%);
}
```



NB : Il est possible de regrouper les différentes propriétés d'arrière-plan en une seule qui est `background` qui aura les valeurs vues précédemment espacées par un espace blanc. Exemple: `background: #FFFF00 url("images/footerlogo.png") no-repeat;`

III.3.2. Les polices

a) Propriétés de la famille font

Les propriétés CSS qui agissent sur la police commencent par font. Ils servent à mettre en forme le contenu texte d'un document.

Propriété font-size

La propriété font-size permet de spécifier la taille de la police. Si cette propriété n'est pas spécifiée alors la taille de la police vaut par défaut 12 points (qui correspond à l'index 3 en HTML).

La taille de la police peut être absolue ou relative et peut s'exprimer de plusieurs manières:

- **L'unité px:** Il s'agit d'une taille absolue. px désigne pixel. La taille de la police désigne la hauteur des caractères en pixel.
- **L'unité pt:** Là encore c'est une taille absolue. pt désigne point. C'est une unité connue en typographie. 1 point mesure 1/72 pouces.
- **Les unités conventionnelles:** Là encore ce sont des tailles absolues très connues, je parle de cm, mm, in (pour inch ou pouce)...
- **L'unité em:** em est une unité relative. 1em désigne la taille normale. Si vous voulez agrandir la taille alors il faut déclarer une valeur supérieure à 1 (par exemple 1.2em) et inversement.
- **L'unité %:** Le pourcentage est également une taille relative. 100% désigne la taille normale et 80% une taille plus petite.

```
h1 {  
  font-size: 36pt;  
}
```

Ce style appliquera la taille 36pt aux titres <h1>. Par conséquent ils auront l'air plus grand car ils mesurent 24pt normalement.

🚦 Propriété font-style

La propriété font-style permet d'écrire un texte en italique. Par défaut elle vaut la valeur normal qui n'a aucun effet sur le texte. La valeur italic incline le texte.

```
span {
  font-style: italic;
}
```

🚦 Propriété font-variant

La propriété font-variant permet de convertir le texte en petites majuscules. Par défaut elle vaut la valeur "normal". Si la valeur "small-caps" est déclarée alors la police sera transformée.

```
h1 {
  font-variant: small-caps;
}
```

TITRE H1 CONVERTI EN PETITES MAJUSCULES.

Les petites majuscules ont la forme des lettres majuscules et la taille des minuscules.

🚦 Propriété font-weight

La propriété font-weight désigne la graisse de la police. Elle peut prendre plusieurs valeurs (vue que la graisse a des niveaux) mais en pratique y'en a que deux qui sont souvent utilisée à savoir bold qui met le texte en gras (comme le fait la balise) et normal qui désactive le gras si le texte l'était déjà (cas des balises <h1>, <h2>... par exemple).

```
h1 {
```

```
font-weight: normal;
}
```

Titre h1 sans graisse.

Propriété font-family

La propriété font-family désigne la police d'écriture du texte. Elle accepte comme valeur le nom de la police.

```
body {
  font-family: verdana;
}
```

Ce qui signifie que tous les textes inclus dans la balise <body> seront écrits en police verdana. Si le nom de la police contient des espaces alors il faut la déclarer entre des guillemets.

```
body {
  font-family: "Century Gothic";
}
```

Bien entendu, la condition pour que la police s'exécute correctement est qu'elle soit disponible sur le système du client. Mais imaginons un instant que la police choisie n'est pas disponible. Dans ce cas, le navigateur chargera la police par défaut qui est "Times New Roman" et qui n'est pas du tout adaptée aux sites Web. La solution consiste donc à déclarer une liste de polices et laisser le navigateur la parcourir jusqu'à ce qu'il rencontre une police disponible.

```
body {
  font-family: "Century Gothic", verdana, sans-serif;
}
```

Dans ce cas, le navigateur cherchera la police "Century Gothic", si elle est disponible il la chargera sinon il passera à "Verdana" etc...

NB : N'oubliez pas de séparer les noms de la police par une virgule (,).

Vous avez sans doute aperçu "sans-serif". En effet, il ne s'agit pas d'une police normale mais une famille générique qui regroupe plusieurs polices. Donc, au lieu de lister beaucoup de polices, on déclare la première, la deuxième, et pourquoi pas la troisième, mais après on déclare la famille générique et laisser le navigateur sélectionner la première police disponible et appartenant à cette famille si les trois premières ne sont pas trouvables.

Les polices génériques

Comme nous avons vu dans le dernier exemple, sans-serif représente une police générique (ou famille générique) regroupant plusieurs polices qui ont des caractéristiques communes.

Il existe plusieurs familles génériques mais sur le Web nous n'avons recours qu'à trois d'entre elles:

- **Famille de police avec empattement (Sérf):** Ce sont des polices qui affichent les caractères avec des terminaisons qui dépassent du caractères. C'est comme des pattes sur lesquels le caractère repose. C'est le cas de "Times New Roman" par exemple. Le nom CSS de cette famille est serif (sans accent).
- **Famille de police sans empattement (Sans Sérf):** Ce sont des polices qui n'affichent pas les terminaisons comme les Sérf. On les appelle aussi des polices à bâtons. Il sont les plus adaptés aux pages Web. Exemple: "verdana" ou "arial". Le nom CSS de cette famille est sans-serif.
- **Famille de police à chasse fixe (Monospace):** Ce sont des polices qui s'étendent sur la même largeur quelque soit le caractère. C'est une police adaptée pour afficher du code source sur une page Web par exemple. Les

polices "Courier" et "Courier New" sont les plus connues de cette famille. Le nom CSS correspondant est monospace.

```
<h1 style="font-weight: normal; font-family: serif">MIME</h1>
```

```
<h1 style="font-weight: normal; font-family: sans-serif">MIME</h1>
```

```
<h1 style="font-weight: normal; font-family: monospace">MIME</h1>
```

Code source
Code source
Code source

Les polices embarquées

On les appelle polices embarquées ou polices en ligne. Il s'agit d'une police qui est envoyée au client par le serveur pour s'en servir durant l'exécution d'une page Web. En effet, le navigateur repose sur les polices installées sur le système, mais cette pratique limite l'utilisation des polices spéciales qui, souvent, ne sont pas disponibles chez la plupart des clients. La solution est donc d'envoyer la police désirée au client pour s'en servir. C'est possible en CSS grâce à la règle @font-face.

Il existe de nombreuses règles en CSS nommées règles-at (pour règle @). Ils commencent par le caractère arobas @. L'une de ces règles-at est @font-face qui permet de spécifier une police spéciale à envoyer au navigateur pour qu'il affiche le texte tel que l'on souhaite.

La syntaxe ressemble à ceci :

```
@font-face {
  font-family: Nom_de_la_police;
  src: url("Chemin_de_la_police_sur_le_serveur");
}
```

- **Nom_de_la_police:** désigne le nom que l'on veut attribuer à la police. Ce nom sera utilisé plus tard dans les propriétés font-family.
- **Chemin_de_la_police_sur_le_serveur:** désigne, comme son nom l'indique, le chemin (absolu ou relatif) de la police (qui est disponible sur le serveur).

Exemple:

```
@font-face {  
  font-family: htower;  
  src: url("fonts/htower.ttf");  
}  
  
h1 {  
  font-weight: normal;  
  font-family: htower;  
}
```

Police embarquée

Vous avez remarqué l'extension de la police? Il s'agit de TTF c'est le format le plus populaire, mais elle n'est pas supporté par tous les navigateur. La solution consiste donc à convertir la police en plusieurs formats pris en charge par les navigateurs et qui sont: TTF, WOFF et SVG, sans oublier le format EOT connu par Internet Explorer.

Si on considérait que la police "htower" de l'exemple précédent a été convertie à toutes ces formats, la règle @font-face ressemblerait à ceci:

```
@font-face{  
  font-family:"htower";  
  src:url("htower.eot");  
  src:url('htower.woff') format("woff"),
```

```

url("htower.svg") format("svg"),
url("htower.ttf") format("truetype");
}

```

Comme Internet Explorer fait toujours l'exception, on a séparé sa déclaration src avec le format EOT. Les trois autres déclarations sont séparées par la virgule. De cette manière le navigateur parcourra la liste jusqu'à ce qu'il trouve le format convenable.

NB : La déclaration format est facultative. Elle indique au navigateur le format de la police. Cependant, l'extension suffit pour en informer.

III.4. STYLES DE TEXTES, MARGES, DIMENSIONS ET LISTES

III.4.1. La famille text

a) Propriété text-align

Comme son nom l'indique, la propriété text-align permet d'aligner le texte au sein de son conteneur. Les différentes valeurs que l'on peut lui attribuer ne vous sont pas étrangères:

- **left:** C'est la valeur par défaut, elle permet d'aligner le texte (ou d'autres objets comme les images) à gauche du conteneur.
- **right:** Pour placer le contenu à droite.
- **center:** Sert à center le contenu.
- **justify:** Permet de justifier le contenu en l'étalant sur toute la longueur de la ligne. Nous aurons donc les lignes (d'un paragraphe par exemple) qui finissent toutes à la même position.

b) Propriété text-indent

La propriété text-indent permet d'appliquer une indentation (alinéa) aux balises de type block (comme <section>, <aside>, <p>, <div>...). La première ligne de

la balise qui reçoit le style marquera un retrait par rapport aux autres lignes. C'est ce que l'on appelle l'indentation. La valeur de `text-indent` s'exprime en pixel.

c) Propriété `text-decoration`

La propriété `text-decoration` permet de souligner, surligner ou barrer un texte. Voici les différentes valeurs possibles:

- **`underline`**: Le texte est souligné (comme le ferait la balise HTML `<u>`).
- **`overline`**: Le texte est surligné. La ligne se trouvera donc en dessus du texte, mais c'est rarement utilisé.
- **`line-through`**: Le texte est barré. La ligne passe à travers le texte comme le ferait la balise HTML `<s>`).
- **`none`**: Aucune décoration ne sera appliquée au texte (toutes les lignes seront retirées).

d) Propriété `text-transform`

La propriété `text-transform` permet de convertir le texte en minuscule, majuscule ou capitalisé. Les différentes valeurs sont:

- **`lowercase`**: Permet de convertir tous les caractères en minuscule.
- **`uppercase`**: Permet de convertir tous les caractères en majuscule.
- **`capitalize`**: Permet de convertir le premier caractère de chaque mot en majuscule et laisser les autres sans transformation.

NB : Il ne faut pas confondre `text-transform:uppercase` avec `font-variant:small-caps`. La première converti le texte en majuscule et la deuxième le convertit en petites majuscules.

e) Propriété `letter-spacing`

Bien que cette propriété ne commence pas par le préfixe `text`, mais elle fait parti de la famille `text`. La propriété `letter-spacing` permet d'appliquer un espacement entre les caractères d'un texte. Cet espacement peut être exprimé en pixel.

Exemple

Code HTML:

```

<h1>Cours de CSS</h1>
<h2>Résumé</h2>
<div>
  CSS (Cascading Style Sheets) ou
  <span>feuilles de style en cascade</span> est un langage de
style simple
  et puissant. Il permet de donner du look à votre page Web
et la rendre
  ainsi présentable.<br />
  Si HTML sert à intégrer du contenu, le CSS quant à lui,
applique du design
  à ce contenu. Il couvre tous les aspects visuelles d'une
page Web,
  commençant de la simple coloration des objets et allant
jusqu'aux et
  transformations complexes de ceux ci et le responsive
design.
</div>

```

Code CSS:

```

body {
  font-family: verdana, arial, sans-serif;
}
h1 {
  text-align: center;
  letter-spacing: 4px;
  color: #ffffff;
  background-color: #000000;
}
h2 {

```

```

    text-transform: uppercase;
}
div {
    text-indent: 20px;
    text-align: justify;
    font-size: 10pt;
}
span {
    text-decoration: underline;
}

```

Cours de CSS

RÉSUMÉ

CSS (Cascading Style Sheets) ou feuilles de style en cascade est un langage de style simple et puissant. Il permet de donner du look à votre page Web et la rendre ainsi présentable.

Si HTML sert à intégrer du contenu, le CSS quant à lui, applique du design à ce contenu. Il couvre tous les aspects visuelles d'une page Web, commençant de la simple coloration des objets et allant jusqu'aux et transformations complexes de ceux ci et le responsive design.

f) Marge externe: famille margin

Pour exprimer les marges en CSS on fait appelle à la propriété margin. Sa valeur est exprimée en pixel. Si par exemple on déclare le style margin:10px, cela signifie que l'objet auquel on l'a appliqué aura une marge qui mesure 10px tout au tour (en haut, en bas, à droite et à gauche).

Pour spécifier une marge différente de chaque coté on fait donc appel aux propriétés suivantes:

- **margin-left:** Signifie la marge gauche.
- **margin-top:** Signifie la marge supérieure.
- **margin-right:** Signifie la marge droite.
- **margin-bottom:** Signifie la marge inférieure.

Supposons que nous avons le style suivant:

```
img {  
  margin: 10px;  
  margin-bottom: 20px;  
}
```

La première propriété signifie que l'image aura une marge de 10px tout autour et la deuxième appliquera une marge inférieure de 20px. Si la propriété `margin-bottom` n'a pas été déclarée, alors la marge inférieure aura aussi la valeur 10px, mais le fait de la définir donne à celle-ci la valeur 20px. L'image aura donc une marge inférieure de 20px car la propriété qui l'applique est plus explicite.

g) Marge interne: famille padding

Si `margin` applique une marge autour de l'objet, `padding` l'applique à l'intérieur de celui-ci. La propriété `padding` applique une marge interne aux balises de types conteneur comme `<header>`, `<footer>`, `<div>`...

Comme pour la propriété `margin`, `padding` applique une marge interne exprimée en pixel de tous les côtés de l'objet. Pour spécifier des valeurs différentes aux marges internes on fait appel aux propriétés suivantes:

- **padding-left**: Signifie la marge interne gauche.
- **padding-top**: Signifie la marge interne supérieure.
- **padding-right**: Signifie la marge interne droite.
- **padding-bottom**: Signifie la marge interne inférieure.

h) Largeur et hauteur

Seules les balises de type block peuvent être redimensionnées. Les balises inline ne le permettent pas à l'exception des images et champs de formulaires.

Pour exprimer la largeur et la hauteur on fait appel aux propriétés `width` et `height`. Elles sont exprimées en pixels, pourcentage ou en unités conventionnelles (cm,

mm, in...). On peut aussi spécifier la largeur ou la hauteur minimales avec les propriétés `min-width` et `min-height` ou maximales grâce à `max-width` et `max-height`.

La largeur minimale permet à l'objet d'avoir une largeur supérieure ou égale à celle spécifiée. Si le contenu est plus grand que l'objet, alors la largeur de celui-ci gagne en taille et si il est plus petit, alors l'objet ne passe pas en dessous de la largeur minimale spécifiée. De même pour la hauteur minimale.

Le même principe s'applique sur la largeur et la hauteur maximales.

Exemple

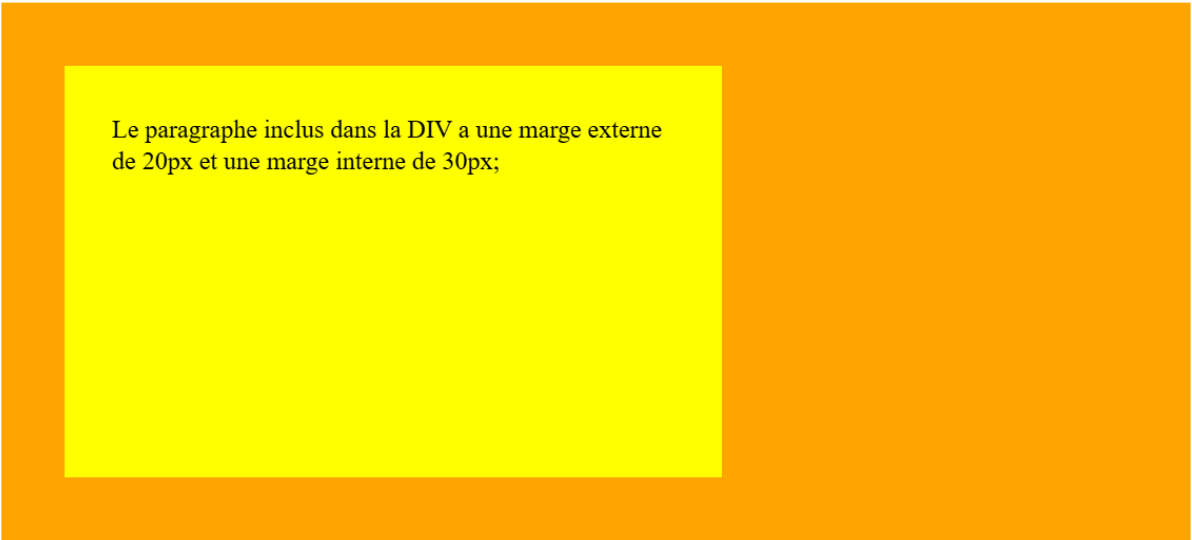
Code HTML

```
<div>
  <p>
    Le paragraphe inclus dans la DIV a une marge externe de
    20px et une
    marge interne de 30px;
  </p>
</div>
```

Code CSS:

```
div {
  background-color: orange;
  padding: 20px;
}

p {
  margin: 20px;
  padding: 30px;
  background-color: yellow;
  width: 50%;
  height: 200px;
}
```



Le paragraphe inclus dans la DIV a une marge externe de 20px et une marge interne de 30px;

Vous avez probablement remarqué que la marge externe de la balise `<p>` constitue la marge interne de la balise `<div>`, car la première est incluse dans la deuxième.

Une autre remarque saute aux yeux; la largeur de la balise `<p>` est supérieure à 50% décrite en CSS. C'est normal car elle mesurerait exactement 50% si elle n'avait pas de padding, sinon la valeurs de padding sera ajoutée à la largeur appliquées (dans ce cas 60px on été ajoutés à la largeur: 30px du padding de gauche et 30px du padding de droite).

i) Les listes

Les listes créées à l'aides des balises `<ul>` ou `<ol>` peuvent être modifiées en CSS. En plus des polices, marges et autres propriétés qui peuvent leur donner un look différent, il existe trois propriétés qui sont spécialement conçues pour elles.

Propriété `list-style-type`

La propriété `list-style-type` permet de définir le type de puce des éléments de la liste. Elle peut accepter les valeurs vues en HTML (disc, circle et square).

Cependant, même si on a défini la liste non ordonnée (à l'aide de la balise `<ul>`) on peut quand même lui donner l'apparence d'une liste ordonnée comme si elle était définie avec la balise `<ol>`) grâce aux valeurs suivantes:

- **decimal**: pour les nombres décimaux commençant de 1.
- **decimal-leading-zero**: pour les nombres décimaux avec zéro initial commençant de 01.
- **upper-roman**: pour les nombres romains majuscules commençant de I.
- **lower-roman**: pour les nombres romains minuscules commençant de i.
- **lower-greek**: pour la numération grecque.
- **lower-alpha**: pour les lettres de l'alphabet minuscules.
- **upper-alpha**: pour les lettres de l'alphabet majuscules.

Propriété list-style-image

Si on veut définir nos propres puces, autant le faire avec des images. La propriété list-style-image.

```
list-style-image: url("chemin_de_l_image");
```

Propriété list-style-position

La propriété list-style-position permet de définir la position de la puce par rapport à la boîte contenant les éléments de la liste. Elle peut avoir deux valeurs: inside et outside.

Exemple:

Code HTML:

```
<ul>
  <li>Premier élément</li>
  <li>Deuxième élément</li>
  <li>Troisième élément</li>
</ul>
```

Code CSS:

```
ul {  
    list-style-type: lower-greek;  
}
```



α. Premier élément
β. Deuxième élément
γ. Troisième élément

III.5. LES BORDURES

Une bordure et un cadre qui entoure des objets comme les images, cellules d'un tableau, conteneurs... CSS offre des styles qui cassent le rendu par défaut appliqué aux bordures à l'aide du HTML par le biais de propriétés que l'on peut personnaliser à notre guise, ce qui offre des possibilités illimitées pour avoir un design optimal.

III.5.1. Propriété border-color

La propriété border-color permet de définir la couleur de la bordure. Elle peut être définie par son nom (anglais), le code hexadécimal, le code décimal (rgb) ou le code décimal avec prise en charge de la transparence (rgba).

Exemple:

```
border-color: #0000ff; /* applique la couleur bleue à la bordure */
```

III.5.2. Propriété border-width

La propriété border-width permet de définir l'épaisseur de la bordure. Elle est exprimée en pixel (l'unité la plus appropriée). Cependant on peut recourir à d'autres unités comme cm, mm, in...

Exemple:

```
border-width: 1px; /* applique une bordure qui fait 1 pixel  
d'épaisseur */
```

III.5.3. Propriété border-style

La propriété border-style permet de définir le style de la bordure. Le style de la bordure définit plusieurs formes grâce aux valeurs que l'on peut résumer ci-après:

- **solid**: pour une bordure solide, c'est à dire, tracée sous forme de trait continu.
- **dotted**: pour une bordure pointillée.
- **dashed**: pour une bordure constituée de tirets.
- **double**: la bordure est doublée (deux traits parallèles).
- **ridge**: donne une bordure en relief.
- **groove**: pour une bordure rainurée.

- **inset**: pour une bordure en relief vers l'intérieur.
- **outset**: pour une bordure en relief vers l'extérieur.
- **none**: aucune bordure n'est appliquée.
- **inherit**: hérite le même style de bordure que l'élément parent (conteneur).

Exemple:

```
border-style: solid;
```

Si on veut avoir des styles différents pour chacune des quatre bordures appliquées autour de l'objet, alors on remplace le préfixe border par:

- **border-top**: qui désigne la bordure supérieure
- **border-right**: qui désigne la bordure droite
- **border-bottom**: qui désigne la bordure inférieure
- **border-left**: qui désigne la bordure gauche

Exemple:

```
border-top-style: solid;
border-right-style: dotted;
border-bottom-style: dashed;
border-left-style: ridge;
```

Cela fait trop de code n'est ce pas? Heureusement on peut remplacer ces quatre lignes par ceci;

```
border-style: solid dotted dashed ridge;
```

En effet, si vous spécifiez plusieurs valeurs séparées par espace le navigateur applique les style dans le sens suivant: bordure supérieure -> bordure droite -> bordure inférieure -> bordure gauche.

On peut même regrouper tous les styles vues pour les bordures dans une seule propriété qui est border comme ceci:

```
border: solid 1px #0000ff;
```

III.5.4.Propriété border-radius

La version 3 de CSS a introduit une nouvelle propriété qui s'applique aux bordures. Il s'agit de border-radius qui applique un arrondi sur les coins de l'objet qui accueille la bordure.

En effet, avant CSS3 les objets susceptibles d'accueillir des bordures comme les cellules d'un tableau, les conteneurs, les champs de formulaires... étaient de forme rectangulaire avec des coins en angle droit. Désormais, et grâce à CSS3, cela a changé. On peut donc avoir des arrondis qui modifient l'apparence classique. La syntaxe ressemble à ceci:

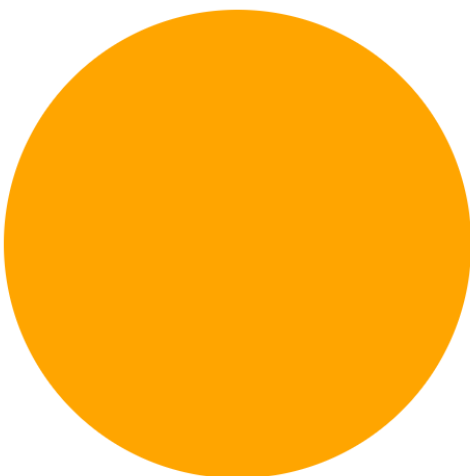
```
div {
  width: 300px;
  height: 300px;
```

```
background-color: orange;  
border-radius: 10px;  
}
```



Essayons maintenant le code suivant:

```
div {  
  width: 300px;  
  height: 300px;  
  background-color: orange;  
  border-radius: 150px;  
}
```



Et oui, nous avons obtenu un cercle. En fait, radius signifie rayon. La propriété `border-radius` arrondi le coin de l'objet de telle sorte à ce qu'il représente le un quart de cercle dont le rayon mesure la valeur attribuée à la propriété. Dans l'exemple précédent, nous avons déclaré un rayon égal à la moitié de la largeur et la hauteur de la balise `<div>`. On a donc appliqué 4 quarts de cercle sur tous les coins. Réunis, ils forment un cercle parfait. Super non?

Si on veut appliquer des arrondis différents à chaque angle d'un objet on procède ainsi:

```
border-radius: Apx Bpx Cpx Dpx;
```

A, B, C et D représentent respectivement les valeurs des arrondis appliquées aux angles: supérieur gauche, supérieur droit, inférieur droit et inférieure gauche.

Dans le sens horaire.

Exemple

Code HTML:

```
<div><h1>DIV stylée</h1></div>
```

Code CSS:

```
div {
  height: 100px;
  background-color: #eeeeee;
  text-align: center;
  padding: 20px;
  border: solid 5px #bbbbbb;
  border-radius: 10px;
  font-family: verdana;
}
```

DIV stylée

III.6. GESTION DES POSITIONS ET TRANSFORMATION DES ELEMENTS

6. Responsive Design

Le responsive design permet d'adapter le site aux écrans.

a) Media queries

```
@media (max-width: 600px) {  
  
  body { background-color: lightblue; }  
  
}
```

b) Mobile first

On commence par styliser pour les petits écrans avant les grands.

Travaux Pratiques (TP)

TP 1 : Styliser une page HTML

Appliquer des couleurs, marges et polices.

TP 2 : Créer un menu

Utiliser Flexbox pour créer un menu horizontal.

TP 3 : Page responsive

Adapter la page pour mobile et ordinateur.

Objectif du chapitre

À la fin de ce chapitre, l'étudiant doit être capable de :

- styliser une page HTML avec CSS
- comprendre le box model
- utiliser Flexbox
- créer une page responsive simple